Contexto de Negocio: Infraestructura de Logística Autónoma

La corporación Sky-Delivery S.A.

Actualmente se encuentra en una fase crítica de despliegue tecnológico. Por ello es imperativo que la arquitectura de software que soporte su flota de drones de reparto sea robusta, escalable y esté diseñada bajo el rigor del Paradigma Orientado a Objetos (POO). No se admitirán soluciones basadas en lógica procedimental; se exige un diseño que mitigue la redundancia mediante el encapsulamiento y un desacoplamiento efectivo entre componentes.

El sistema debe coordinar de manera precisa las siguientes entidades:

● Drones: Unidades autónomas que gestionan internamente niveles de energía y telemetría (coordenadas GPS).

● Paquetes: Entidades con dimensiones físicas y peso que condicionan la autonomía del transporte.

● Centros de Despacho: Nodos logísticos responsables de la orquestación y asignación de rutas. Como arquitectos, su misión es garantizar que el diseño soporte la complejidad operativa de una flota de alta disponibilidad, asegurando que la transición entre el modelo teórico y la ejecución en Máquina Virtual sea impecable.

2. UML

DIAGRAMA CLASES

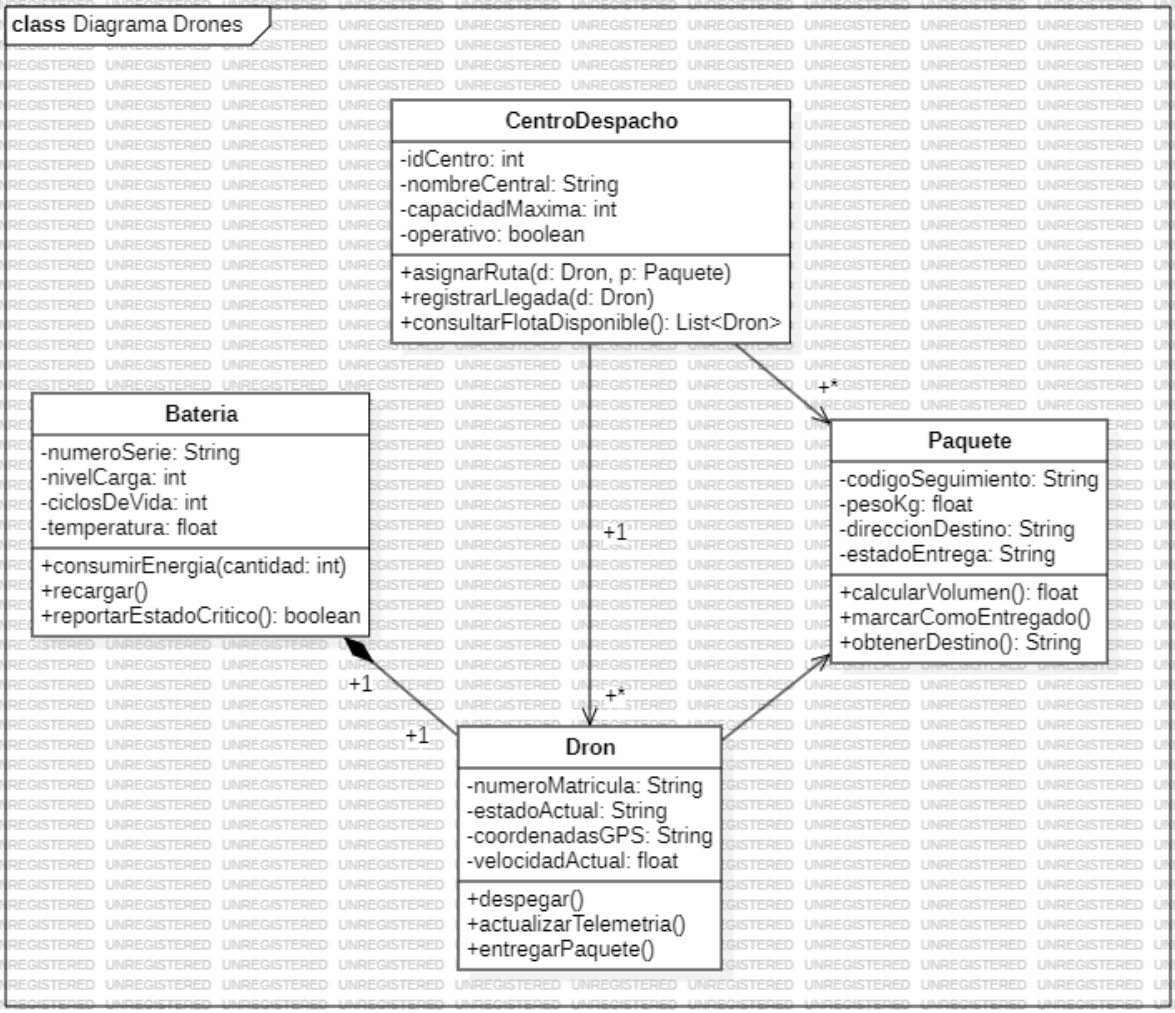
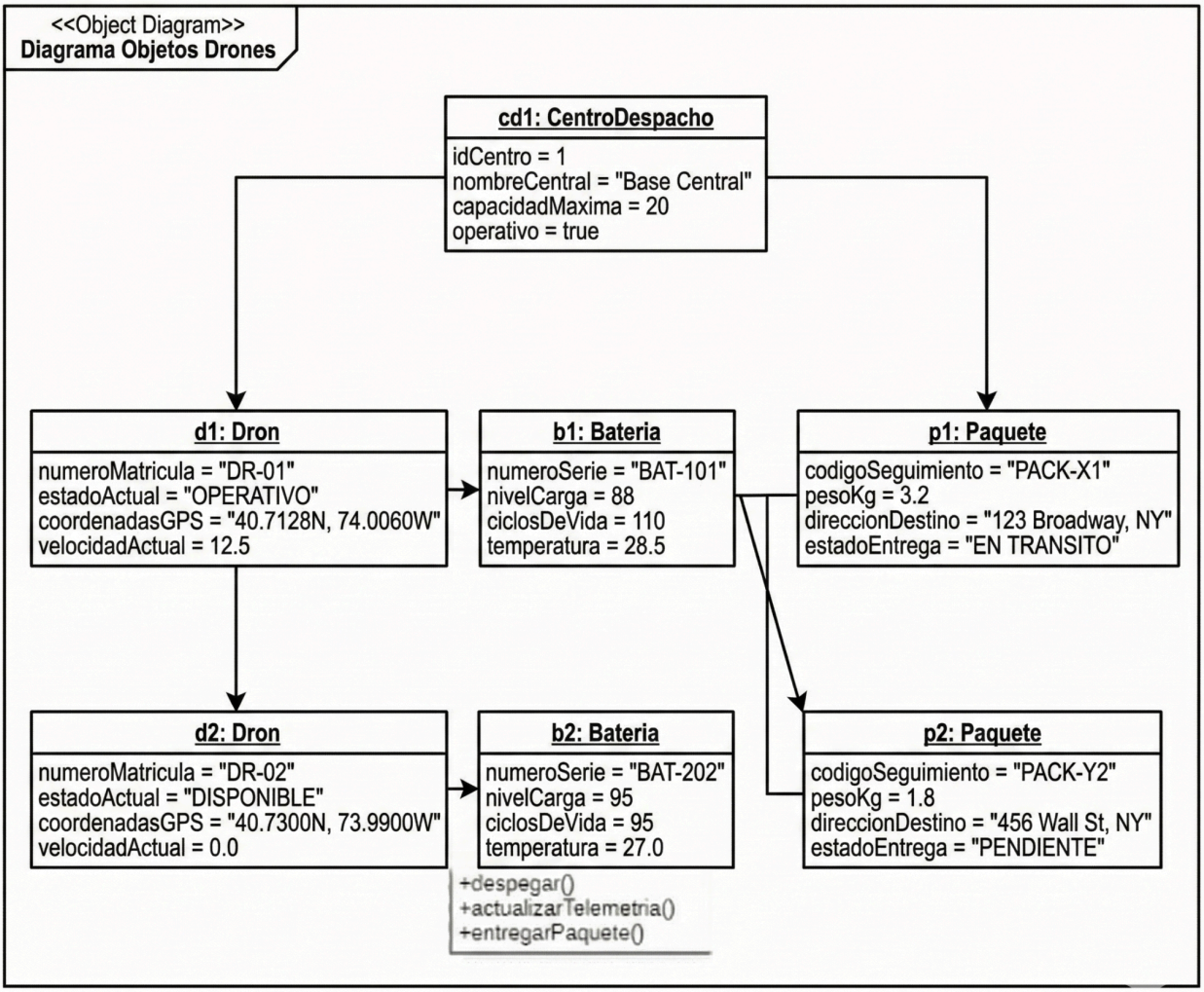


DIAGRAMA DE OBEJTOS



3. Desafíos Técnicos de Arquitectura y Memoria (UVA 2)

3.1 Gestión en la Máquina Virtual (VM)

1- La eficiencia del sistema de drones depende de cómo la JVM administra los recursos en tiempo de ejecución. Por ejemplo, **Stack** almacena las variables de referencia (ej. Drone d1). Es una memoria de acceso rápido y tamaño limitado que sigue un flujo LIFO.
Cuando el hilo de ejecución sale del bloque (scope), la variable de referencia se libera automáticamente.

Heap en cambio, es el espacio dinámico donde está la instancia real del objeto (ej. Nz ew Drone()). Si un objeto en el Heap pierde todas sus referencias en el Stack, queda huérfano y el Garbage Collector lo marca para su eliminación, liberando la memoria.

2- Ciclo de Vida de la Ejecución:

- Compilación: El código fuente (.java) se traduce a Bytecode (.class), un lenguaje intermedio independiente de la plataforma.

- Verificación y Carga: El Class Loader incorpora las clases y la JVM valida que el bytecode cumpla con las reglas de seguridad.

- Ejecución: El intérprete o el compilador JIT (Just-In-Time) traduce el bytecode a instrucciones de código máquina específicas para el procesador del servidor logístico.

3.2 Tipos de Datos y Wrappers

Para el atributo *"nivelDeBateria"*, se prefiere el tipo primitivo double.
Pues en un sistema de alta disponibilidad con miles de unidades, el primitivo destaca por un superior rendimiento y ahorro de memoria.

Comparativa: Mientras que un Double (Wrapper) permitiría el uso de valores null o métodos utilitarios, su uso implicaría un costo de procesamiento extra por el autoboxing/unboxing. Dado que la telemetría de un drone siempre debe arrojar un valor numérico válido, el double garantiza una lógica de control más fluida y eficiente.

3.3 **Firma y Contrato:**

El método solicitado válida si el drone tiene suficiente batería para transportar el paquete asignado. Retorna `true` si el vuelo puede realizarse, `false` en cualquier caso que lo impida.

Pseudo-código del metódo autorizarVuelo():

```
MÉTODO autorizarVuelo():
   SI nivelDeBateria <= 0 ENTONCES
      RETORNAR false  // sin batería, vuelo imposible
   SI paqueteAsignado ES NULO ENTONCES
      RETORNAR false  // no hay paquete asignado
   SI peso del paquete > capacidadMaximaCarga ENTONCES
      RETORNAR false  // paquete demasiado pesado
   SI nivelDeBateria < (peso del paquete * FACTOR_CONSUMO) ENTONCES
      RETORNAR false  // batería insuficiente para ese peso
   RETORNAR true
```

Instanciación de Flota:

La definición de como se inicializar un arreglo de tipos de referencia para una flota de 50 drones, estaría así:

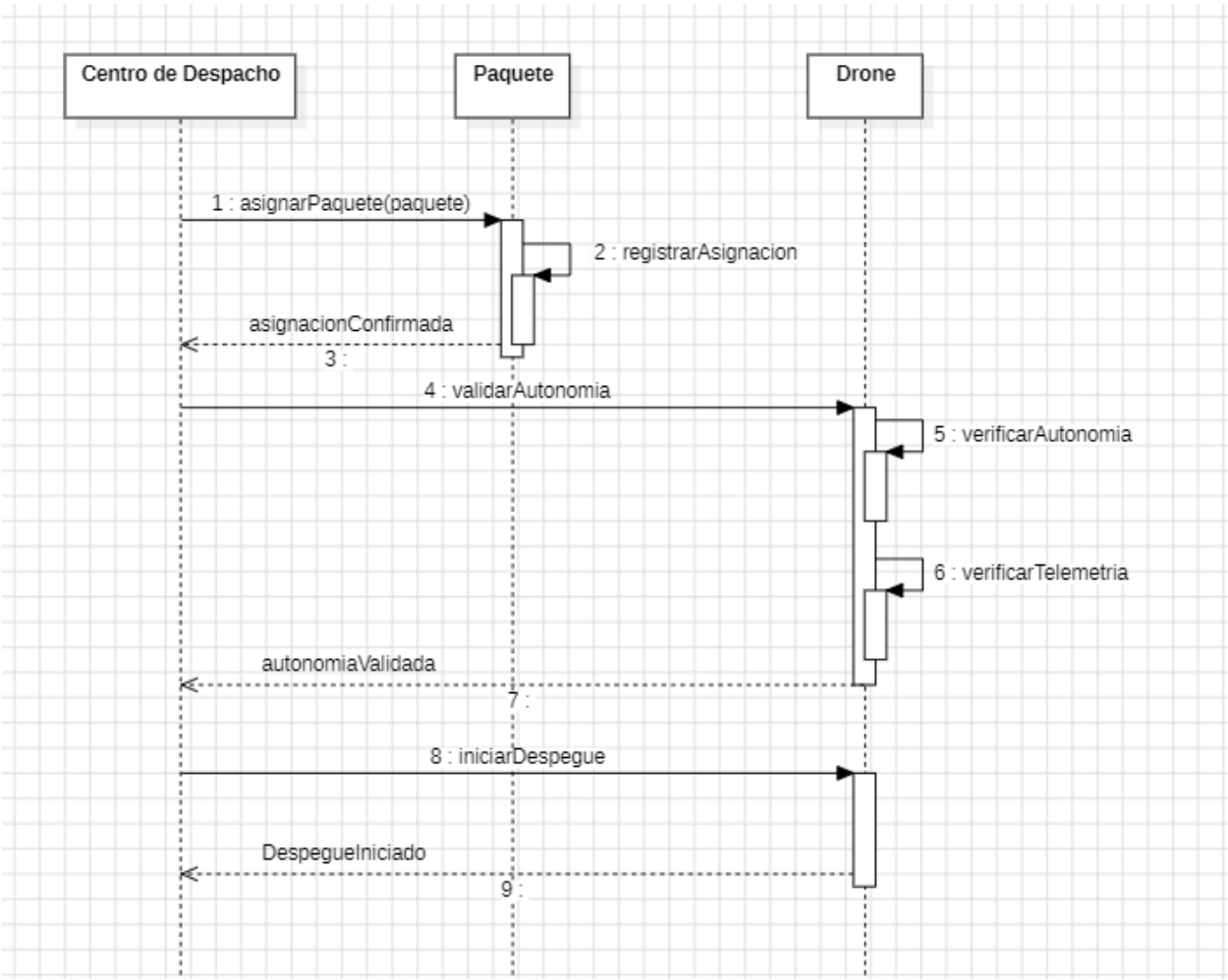
```
DECLARAR flota COMO arreglo de Drone DE TAMAÑO 50

PARA i DESDE 0 HASTA 49:
   flota[i] = CREAR NUEVO Drone(
      id       = "DRONE-" + (i + 1),
      bateria   = 100,
      capacidad = 5.0 kg
   )
FIN PARA
```

Explicación del arreglo:

Declarar el arreglo solo crea el contenedor con posiciones vacías. Sin instanciar cada elemento individualmente con `CREAR NUEVO Drone()`, cualquier intento de usar una posición genera una excepción en tiempo de ejecución, ya que no existe ningún objeto al que apuntar.

4


Centro de Despacho
Paquete
Drone
1 : asignarPaquete(paquete)
2 : registrarAsignacion
asignacionConfirmada
3 :
4 : validarAutonomia
5 : verificarAutonomia
6 : verificarTelemetria
autonomiaValidada
7 :
8 : iniciarDespegue
DespegueIniciado
9 :